

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра дискретной математики и информационных технологий

ТЕМА

КУРСОВАЯ РАБОТА

студента 3 курса 321 группы
направления 09.03.01 — Информатика и вычислительная техника
факультета КНиИТ
Номеровского Максима Сергеевича

Научный руководитель
старший преподаватель

Н. Е. Тимофеева

Заведующий кафедрой
доцент, к. ф.-м. н.

Л. Б. Тяпаев

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические основы	5
1.1 Одноплатный компьютер MangoPi MQ Pro	5
1.2 Ядро Linux	6
1.2.1 Основные понятия	6
1.2.2 Сборка ядра	6
1.2.3 Дерево устройств	6
1.2.4 Интерфейс SPI	7
1.3 Загрузчик U-Boot	8
1.4 Arduino	9
2 Практическая часть	11
2.1 Компиляция ядра Linux	11
2.2 Создание образа системы для MangoPi MQ Pro	11
2.3 Создание программы генератора меандра и программы повто- рителя	11
2.4 Сборка схемы и проверка работы	11
ЗАКЛЮЧЕНИЕ	12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	14
Приложение А Исходный код программы для Arduino	15
Приложение Б Исходный код программы для MangoPi MQ Pro	16

ВВЕДЕНИЕ

В последние годы развитие встраиваемых систем и интернета вещей стимулирует активное использование одноплатных компьютеров на базе операционной системы Linux. Ключевым аспектом создания надёжных аппаратно-программных комплексов является глубокое понимание механизмов взаимодействия ядра ОС с периферийными контроллерами. Интерфейс SPI остаётся одним из наиболее распространённых и эффективных протоколов для обмена данными между микроконтроллерами и одноплатными платформами. Освоение процессов кросс-компиляции ядра Linux, конфигурирования дерева устройств и организации аппаратного взаимодействия с внешними контроллерами, такими как Arduino, имеет высокую практическую ценность для разработки систем сбора данных, промышленной автоматизации и образовательных проектов.

Актуальность данной работы обусловлена необходимостью системного изучения полного цикла создания программного обеспечения для одноплатного компьютера. Сборка ядра Linux и подготовка DTB-файла позволяют гибко адаптировать операционную систему под специфическую аппаратную конфигурацию платы, исключая избыточные модули и оптимизируя использование ресурсов. Кроме того, интеграция одноплатного компьютера с микроконтроллерной платформой для генерации и анализа тестовых сигналов служит эффективным способом верификации корректной работы драйверов, DMA-каналов и пользовательских API.

Целью данной курсовой работы является создание кросс-компилированного ядра Linux[1] и файла дерева устройств для одноплатного компьютера MangoPi MQ Pro[2], а также разработка программного обеспечения для микроконтроллера Arduino[3], реализующего генерацию меандровых сигналов по протоколу SPI, и обеспечение их приёма и воспроизведения в режиме обратной связи на стороне одноплатной платформы. Для достижения поставленной цели в работе будут решены следующие задачи:

1. Выполнить конфигурирование и кросс-компиляцию ядра Linux с включением необходимых драйверов и поддержки требуемых файловых систем.
2. Подготовить исходные файлы дерева устройств, активировать контроллер SPI на соответствующих выводах GPI и скомпилировать DTB-файл.

3. Разработать программу для Arduino, функционирующую в роли SPI-мастера и генерирующую тестовые меандровые последовательности.
4. Реализовать программу на стороне MangoPi MQ Pro для приёма SPI-данных через /dev/spidev и их обратной отправки с последующей верификацией целостности сигналов.
5. Провести аппаратное подключение, тестирование взаимодействия и оценку корректности работы созданной программно-аппаратной связки.

1 Теоретические основы

1.1 Одноплатный компьютер MangoPi MQ Pro

Одноплатный компьютер – это полноценный компьютер, размещённый на одной печатной плате. Он содержит процессор, память, устройства ввода-вывода и другие компоненты, необходимые для работы. В отличие от настольных ПК, одноплатники имеют компактные размеры, низкое энергопотребление и часто используются во встраиваемых системах, IoT-устройствах и образовательных целях.

MangoPi MQ Pro – одноплатный компьютер оснащённый SoC Allwinner D1[3].

Особенности платы:

- Система-на-кристалле Allwinner D1 с процессором XuanTie C906 (архитектура RISC-V) с тактовой частотой до 1 ГГц.
- Оперативная память объёмом 512 МБ или 1 ГБ типа DDR3/DDR3L.
- Порт USB-OTG типа Type-C для подключения к ПК и подачи питания.
- Порт USB-Host типа Type-C для подключения внешних периферийных устройств.
- 40-контактный разъём расширения GPI.
- 24-контактный разъём для подключения цифровых камер или Ethernet-модулей (DVP/RGMII).
- Разъём mini HDMI для вывода видеосигнала на внешние дисплеи.
- Слот для карт памяти формата microSD (TF), используемый в качестве основного загрузочного носителя.
- Встроенный комбинированный модуль беспроводной связи Wi-Fi и Bluetooth RTL8723DS.
- 20-контактный FPC-разъём для подключения дисплеев по интерфейсам DSI, CTP или LVDS.
- Контактные площадки для подключения внешнего аудиовыхода.
- Встроенная печатная антенна для модуля Wi-Fi/Bluetooth.
- Компактные габариты платы размером 6,5х3 см.

1.2 Ядро Linux

1.2.1 Основные понятия

Ядро Linux – центральный компонент операционной системы, расположенный между аппаратным обеспечением и пользовательскими приложениями[1].

Его основные функции:

- Управление аппаратными ресурсами: распределение процессорного времени, управление памятью, координация доступа к устройствам ввода-вывода.
- Обеспечение безопасности и разграничение прав доступа между процессами и пользователями.
- Поддержка сетевых протоколов стека TCP/IP.
- Управление файловыми системами и обеспечение чтения/записи данных.
- Предоставление API системных вызовов для пользовательских программ.

1.2.2 Сборка ядра

Сборка ядра Linux из исходного кода – это компиляция и компоновка нескольких миллионов строк кода на языке C в исполняемый бинарный файл на основе пользовательской конфигурации. Конфигурация хранится в файле `.config` в корне дерева исходников.

Поскольку разработка ведётся на хост-машине с архитектурой x86_64, а целевая платформа - RISC-V, необходим кросс-компилятор. Кросс-компилятор – это специальный компилятор, который позволяет компилировать код для платформы, отличной от той, на которой он выполняется. Для компиляции в RISC-V архитектуру используется специальный набор инструментов с префиксом `riscv64-linux-gnu-`, устанавливаемый из репозитория дистрибутива.

1.2.3 Дерево устройств

Device Tree (DT) – это структура данных, описывающая аппаратную конфигурацию системы для операционной системы. Появление Device Tree решило важную архитектурную проблему: без него каждая плата требовала бы отдельной сборки ядра с жёстко зашитыми адресами регистров и описаниями периферии. Благодаря Device Tree одно и то же бинарное ядро может запускаться на разных платах одного семейства: информация об аппаратуре

передаётся ядру отдельным файлом в момент загрузки, а не компилируется внутрь.

DTS (Device Tree Source) – человекочитаемый текстовый файл с расширением .dts, содержащий описание аппаратуры конкретной платы. Файлы с расширением .dtsi являются включаемыми заголовочными файлами.

DTB (Device Tree Blob) – бинарный скомпилированный файл .dtb, который загружается загрузчиком и передаётся ядру Linux в момент старта.

DTC (Device Tree Compiler) – компилятор, преобразующий DTS-файлы в DTB и обратно.

Основной источник dts – исходный код ядра Linux и репозитории разработчиков одноплатных компьютеров.

1.2.4 Интерфейс SPI

SPI (Serial Peripheral Interface) – синхронный последовательный интерфейс для обмена данными между микроконтроллером (мастером) и периферийными устройствами (ведомыми).

Интерфейс использует четыре сигнальные линии:

- SCLK (Serial Clock) – тактовый сигнал, генерируемый исключительно мастером.
- MOSI (Master Out Slave In) – линия передачи данных от мастера к ведомому.
- MISO (Master In Slave Out) – линия передачи данных от ведомого к мастеру.
- SS / CS (Slave Select / Chip Select) – сигнал выбора ведомого, активный низкий. Для каждого ведомого требуется отдельная линия SS.

Важной особенностью SPI является полнодуплексный характер обмена: при каждой передаче данных от мастера ведомый одновременно возвращает ответ той же длины в битах, даже если этот ответ не несёт полезной информации.

Подсистема SPI в ядре Linux организована в виде многоуровневой архитектуры абстракции. На нижнем уровне располагается аппаратный драйвер контроллера (Host Controller Driver), отвечающий за непосредственное управление конкретным SPI-интерфейсом системы-на-кристалле. Над ним функционирует ядро самой подсистемы SPI, которое предоставляет унифицированный программный интерфейс для протокольных драйверов и обеспе-

чивает централизованное управление очередью транзакций. Третий уровень занимают протокольные драйверы, реализующие специфические алгоритмы обмена данными для отдельных периферийных устройств. Доступ из пользовательского пространства обеспечивается специальным драйвером SPIDEV, который создаёт символьные устройства вида `/dev/spidevX.Y` и позволяет прикладным программам взаимодействовать с шиной SPI посредством стандартных системных вызовов `open`, `read`, `write` и `ioctl`[4].

1.3 Загрузчик U-Boot

Загрузчик (bootloader) – это небольшая программа, которая запускается сразу после включения устройства и отвечает за базовую инициализацию аппаратной части платы (процессор, память, диски), поиск и загрузку в память основной операционной системы (ядра Linux), передачу управления этой операционной системе.

Для одноплатников самым распространенным загрузчиком является U-Boot[6].

Этапы загрузки операционной системы одноплатного компьютера:

1. **Boot ROM.** Процессор выполняет встроенный код базовой аппаратной инициализации и осуществляет поиск вторичного загрузчика на внешнем носителе.
2. **SPL (Secondary Program Loader).** Компактная версия U-Boot, предназначенная исключительно для инициализации контроллера оперативной памяти (DRAM), что необходимо для загрузки полноценного загрузчика.
3. **U-Boot Proper.** Основная версия загрузчика, инициализирующая периферийные интерфейсы и загружающая в память образы ядра Linux, файла дерева устройств (DTB) и компонента OpenSBI.
4. **OpenSBI.** Реализация спецификации SBI, работающая в привилегированном режиме M-mode. Выполняет финальную низкоуровневую инициализацию процессора и подготавливает аппаратное окружение для передачи управления ОС.
5. **Ядро Linux.** После переключения в режим S-mode ядро инициализирует устройства согласно DTB, монтирует корневую файловую систему и запускает первый пользовательский процесс `init`.

U-Boot использует стандарт Extlinux Configuration для определения па-

раметров запуска ядра. Файл `extlinux.conf` размещается в директории `/extlinux/` на разделе `BOOT (FAT32)`.

1.4 Arduino

Arduino – это открытая платформа для прототипирования электронных устройств, основанная на простой в использовании аппаратной и программной среде. По сути, это микроконтроллерный комплект, который позволяет быстро создавать электронные проекты без глубоких знаний в электронике[2].

Процесс программирования Arduino состоит из нескольких этапов. Сначала разработчик пишет код в среде Arduino IDE, используя упрощённый диалект C/C++ с готовыми библиотеками для работы с периферией. Затем среда разработки компилирует код в машинные инструкции для архитектуры AVR, оптимизируя его под ограниченные ресурсы микроконтроллера.

После компиляции происходит загрузка прошивки через USB-порт с использованием встроенного программатора. Как только прошивка успешно загружена, программа начинает выполняться сразу же – микроконтроллер переходит к выполнению кода без необходимости перезагрузки или дополнительных действий.

Основные концепции программирования Arduino:

Скетч (sketch) – так называется программа для Arduino. Каждый скетч состоит из двух обязательных функций:

```
void setup() {  
    // Выполняется один раз при старте  
    // Инициализация пинов, настройка периферии  
}  
  
void loop() {  
    // Выполняется бесконечно в цикле  
    // Основная логика программы  
}
```

Цифровые выводы общего назначения (GPIO) представляют собой универсальные интерфейсные линии, конфигурируемые в различные режимы функционирования с помощью стандартной функции `pinMode()`. В базовой реализации выделяют два основных направления работы: режим вывода, при

котором микроконтроллер формирует заданный логический уровень напряжения на контакте для непосредственного управления внешними исполнительными устройствами, и режим ввода, в котором вывод переходит в состояние высокоимпедансного входа, позволяя вычислительному ядру регистрировать внешние электрические сигналы, поступающие от внешних устройств.

Прерывания (interrupts) – механизм, позволяющий микроконтроллеру оперативно реагировать на внешние или внутренние события без организации постоянного опроса в основном цикле программы. При возникновении прерывания процессор приостанавливает выполнение текущего кода, сохраняет контекст (содержимое регистров и счётчик команд), передаёт управление функции-обработчику, а после её завершения восстанавливает контекст и возобновляет выполнение с точки прерывания.

Различают два основных типа прерываний:

Аппаратные прерывания возникают от внешних устройств и периферии микроконтроллера – сигналов на цифровых пинах, таймеров, UART, SPI, I2C и других коммуникационных интерфейсов.

Программные прерывания инициируются самим кодом через специальные инструкции процессора – они используются для вызова функций операционной системы или приоритетного переключения задач.

Платформа Arduino функционирует в режиме аппаратного SPI-мастера, требуя строгой синхронизации параметров обмена с ведомым устройством: режима работы (Mode 0–3, CPOL/CPHA), частоты тактирования (задаваемой делителем от системных 16 МГц) и порядка передачи битов (MSB/LSB). Линия выбора ведомого (SS/CS) управляется программно через стандартные GPIO-выводы, обеспечивая гибкое управление транзакциями независимо от конфигурации аппаратного контроллера[5].

2 Практическая часть

2.1 Компиляция ядра Linux

2.2 Создание образа системы для MangoPi MQ Pro

2.3 Создание программы генератора меандра и программы повторителя

2.4 Сборка схемы и проверка работы

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был изучен полный цикл разработки программного обеспечения нижнего уровня для встраиваемых платформ на базе операционной системы Linux. Исследование проводилось на примере одноплатного компьютера MangoPi MQ Pro, оснащённого системой-на-кристалле Allwinner D1 с архитектурой RISC-V. Особое внимание уделялось организации надёжного межаппаратного взаимодействия через синхронный последовательный интерфейс SPI, где в качестве внешнего генератора тестовых сигналов использовалась платформа на базе микроконтроллера Arduino.

Основным результатом работы стала успешная адаптация ядра Linux под специфическую аппаратную конфигурацию выбранной платы. Были выполнены конфигурирование и кросс-компиляция ядра с включением необходимых драйверов, подготовка и компиляция бинарного файла дерева устройств (DTB), а также формирование полноценного загрузочного образа операционной системы. На программном уровне реализован механизм приёма и обработки данных по протоколу SPI, что подтвердило корректность настройки контроллера, DMA-каналов и пользовательского интерфейса `/dev/spidev`. Разработанное решение обеспечивает стабильный полнодуплексный обмен данными и может служить надёжной основой для дальнейшего расширения функциональности встраиваемой системы.

Ограничениями выполненной работы являются следующие аспекты. Во-первых, использование режима обратной связи (loopback) для тестирования не позволяет в полной мере оценить работу системы при асинхронном обмене данными с несколькими ведомыми устройствами. Во-вторых, отсутствует оптимизация передачи больших объёмов данных через SPI, что в перспективе может потребовать реализации механизмов DMA и буферизации в пользовательском пространстве. В-третьих, применена упрощённая схема согласования уровней напряжений, которая в промышленных решениях должна быть заменена на специализированные преобразователи логических уровней для обеспечения повышенной надёжности и защиты интерфейсных линий.

Разработанное решение демонстрирует возможность гибкой адаптации ядра ОС под конкретную аппаратную конфигурацию, а также эффективность использования SPI-интерфейса для задач межаппаратного обмена дан-

ными в реальном времени. При этом полученная программно-аппаратная связка предоставляет возможности для тестирования различных сценариев обработки входных сигналов: варьирование параметров ядра, модификация программы-приёмника на стороне MangoPi и изменение алгоритма генерации тестовых последовательностей на стороне Arduino. Такая архитектура позволяет проводить сравнительный анализ производительности, оценивать задержки передачи данных и исследовать устойчивость канала связи при различных нагрузках, что делает решение пригодным не только для образовательных целей, но и для прототипирования более сложных встраиваемых систем, требующих надёжного межаппаратного взаимодействия.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 The Linux Kernel documentation [Электронный ресурс] URL: <https://docs.kernel.org/> (дата обращения: 27.04.2026)
- 2 Arduino documentation [Электронный ресурс] URL: <https://docs.arduino.cc/> (дата обращения: 19.05.2026)
- 3 MangoPi MQ Pro documentation [Электронный ресурс] URL: https://mangopi.org/mangopi_mqpro (дата обращения: 27.04.2026)
- 4 Linux spidev documentation [Электронный ресурс] URL: <https://www.kernel.org/doc/Documentation/spi/spidev> (дата обращения: 27.04.2026)
- 5 Arduino SPI documentation [Электронный ресурс] URL: <https://docs.arduino.cc/language-reference/en/functions/communication/SPI/> (дата обращения: 19.05.2026)
- 6 U-Boot documentation [Электронный ресурс] URL: <https://docs.u-boot.org/en/latest/> (дата обращения: 19.05.2026)

ПРИЛОЖЕНИЕ А

Исходный код программы для Arduino

ПРИЛОЖЕНИЕ Б

Исходный код программы для MangoPi MQ Pro

Курсовая работа "ТЕМА" выполнена мною самостоятельно, и на все источники, имеющиеся в курсовой работе, даны соответствующие ссылки.

подпись, дата

инициалы, фамилия